



ВШЭ

Программная
инженерия

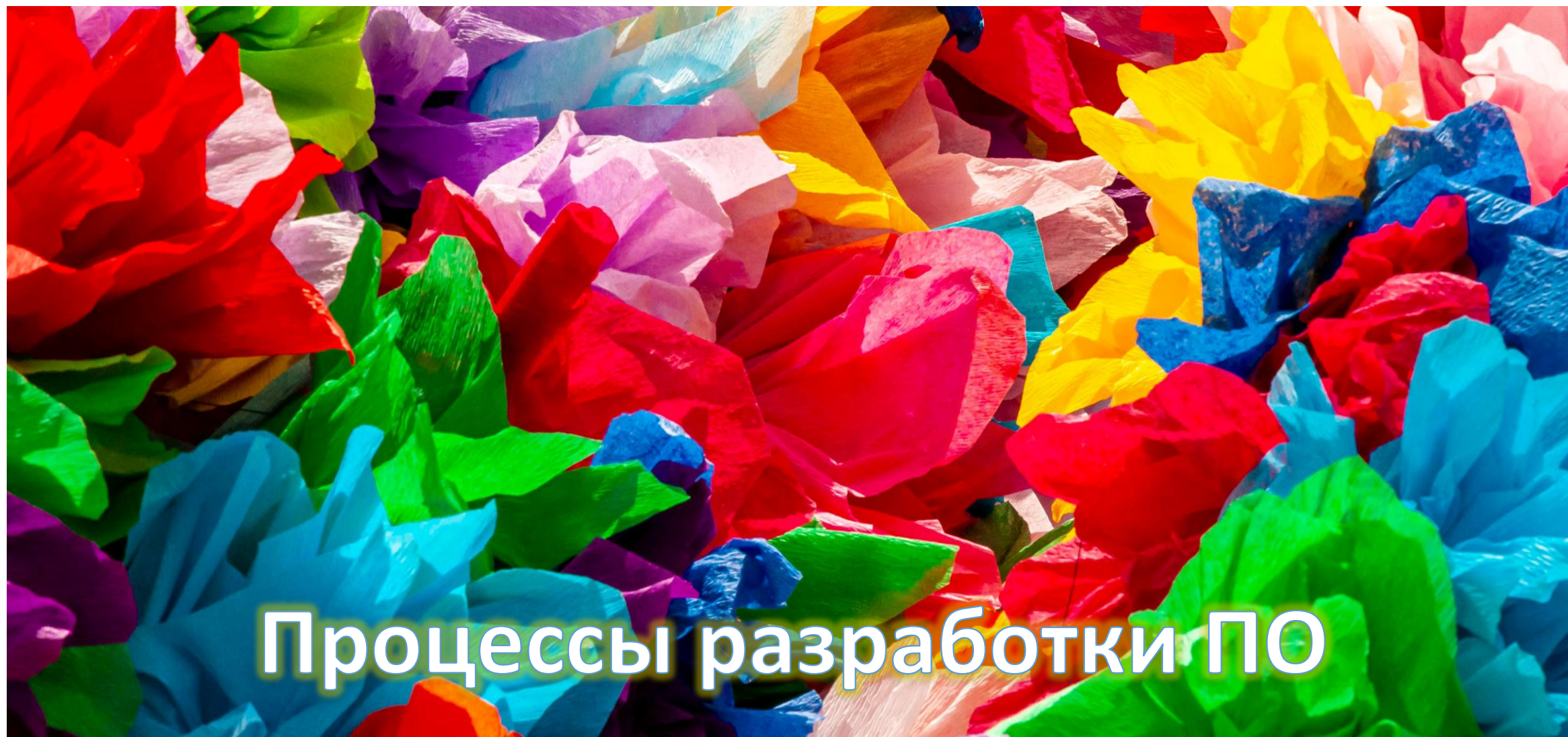
Москва
2026

Лекция № 11.

Безопасная разработка ПО.

DevSecOps

Основы кибербезопасности
Белявский Д.А.



Процессы разработки ПО

Упрощенный процесс разработки ПО

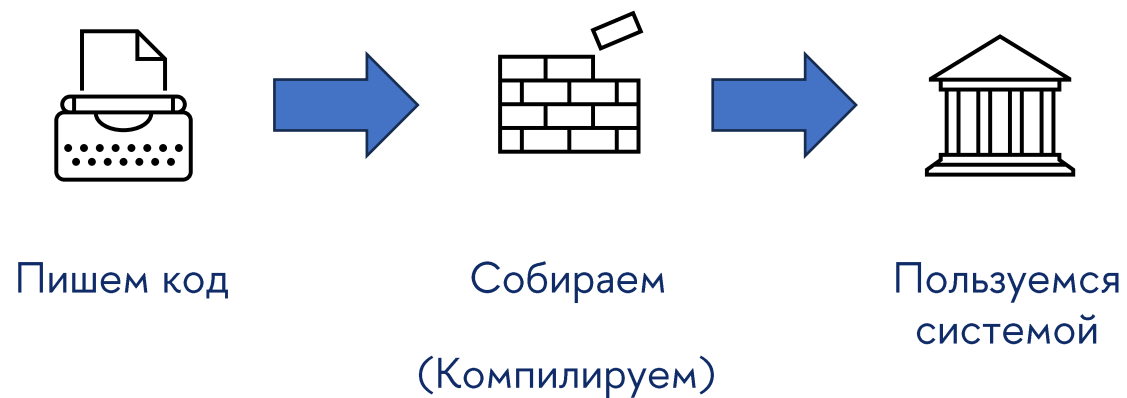


Пишем код

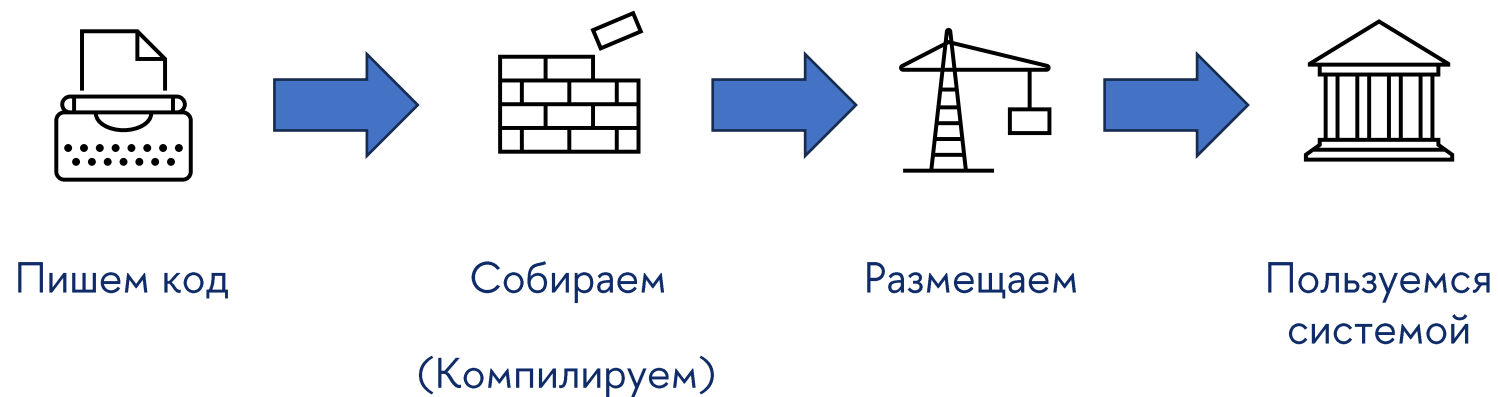


Пользуемся
системой

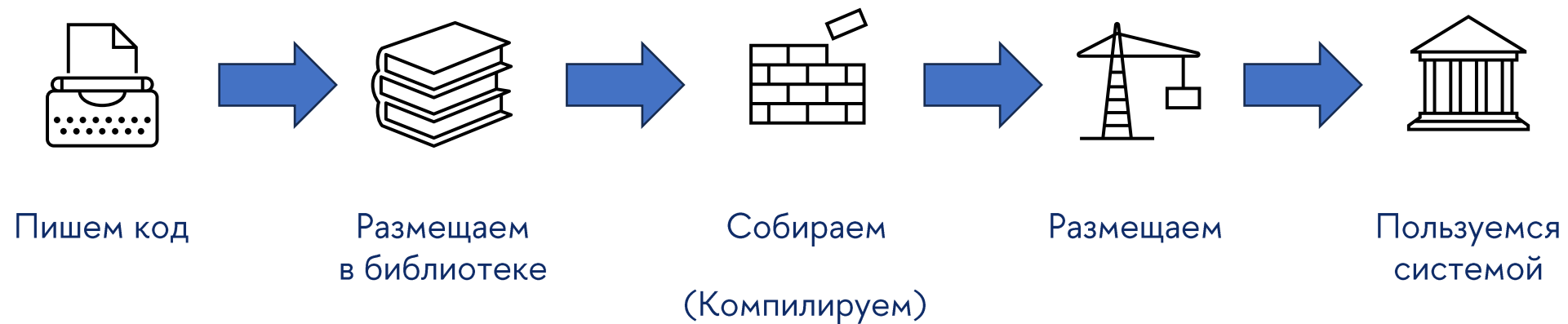
Немного усложненный процесс разработки ПО



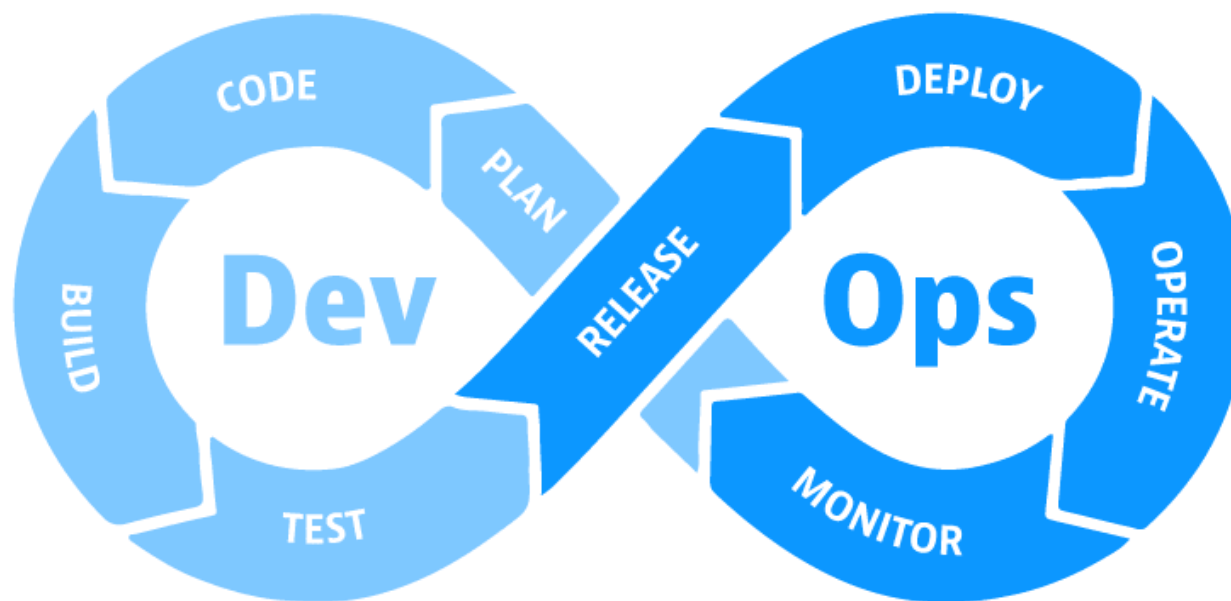
Еще немного усложненный процесс разработки ПО



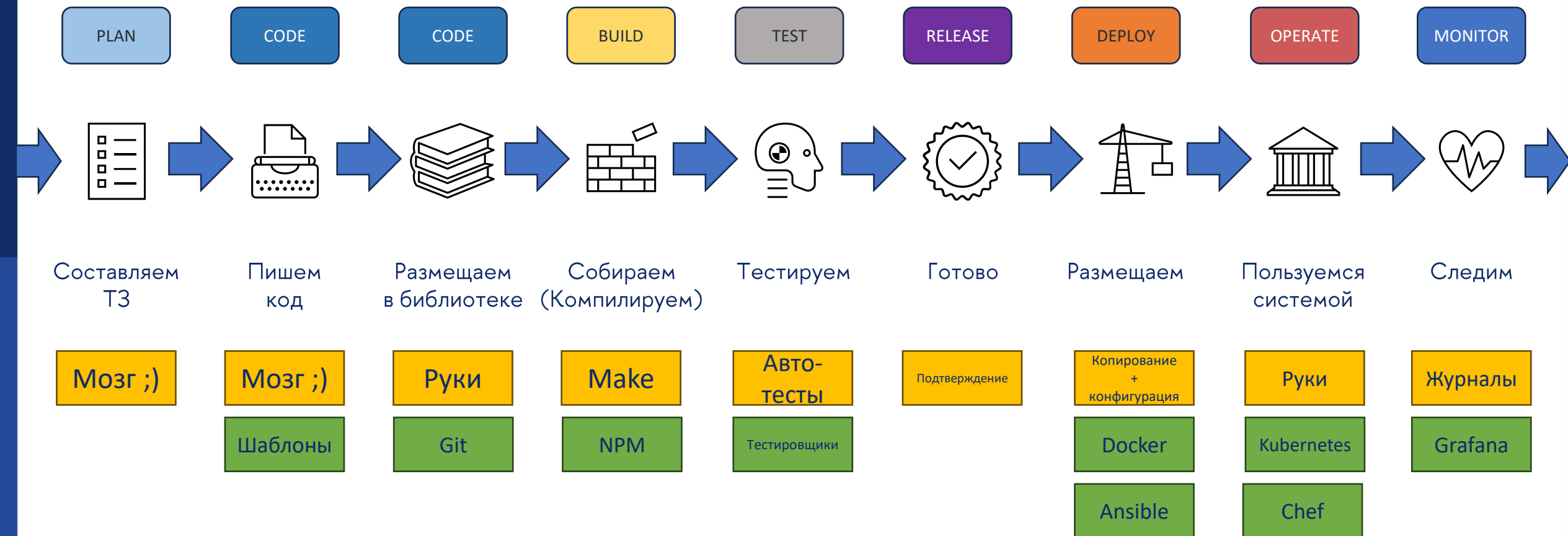
... и еще немного усложненный процесс разработки ПО



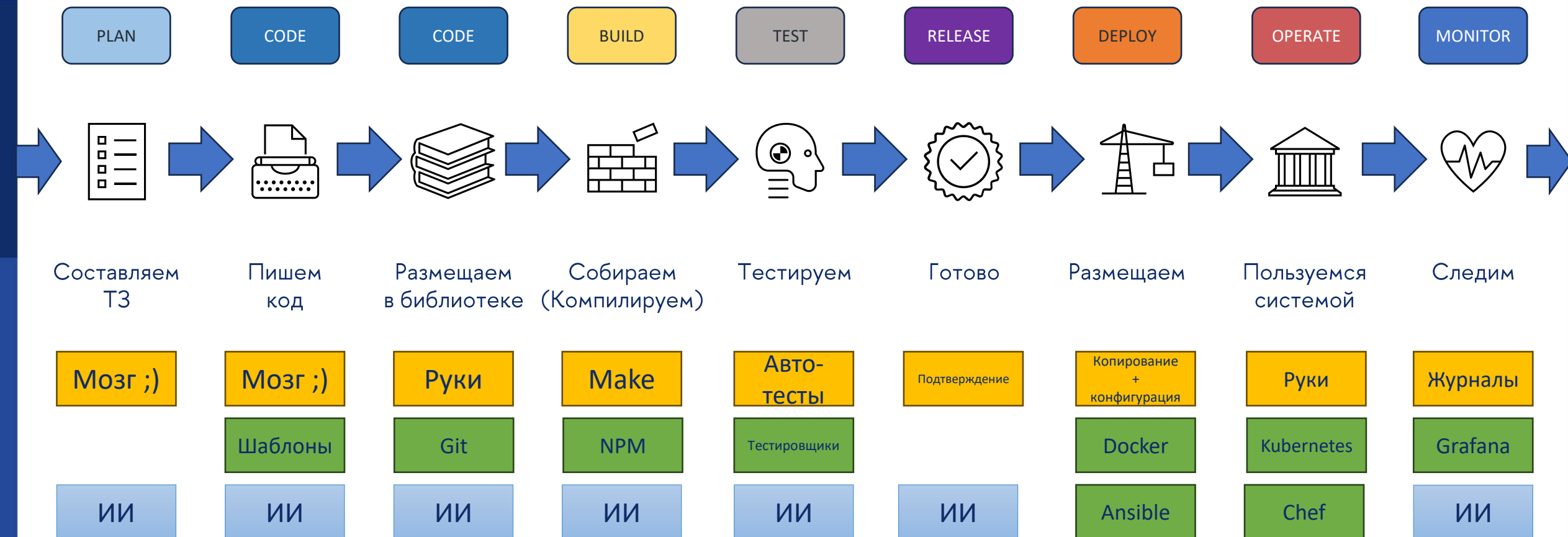
Development + Operations = DevOps



Подробный процесс разработки ПО



Подробный процесс разработки ПО





Безопасная разработка ПО

В любом сложном процессе возникают вопросы, тем более в процессе разработке ПО

Если не контролировать процесс разработки ПО с точки зрения безопасности то можно подарить злоумышленникам:

Уязвимости!

Логин, пароли,
токены в публичных
репозиториях

Использование
уязвимых библиотек
(так как не проверяли)

Сохранение паролей и
ключей при
публикации

Неверная
конфигурация
(misconfiguration)
продуктивных сред

Это только верхушка айсберга.

Почему важно придерживаться требований безопасности при разработке ПО?

70%

Разработчиков пропускают задачи безопасности из-за сжатых сроков сдачи

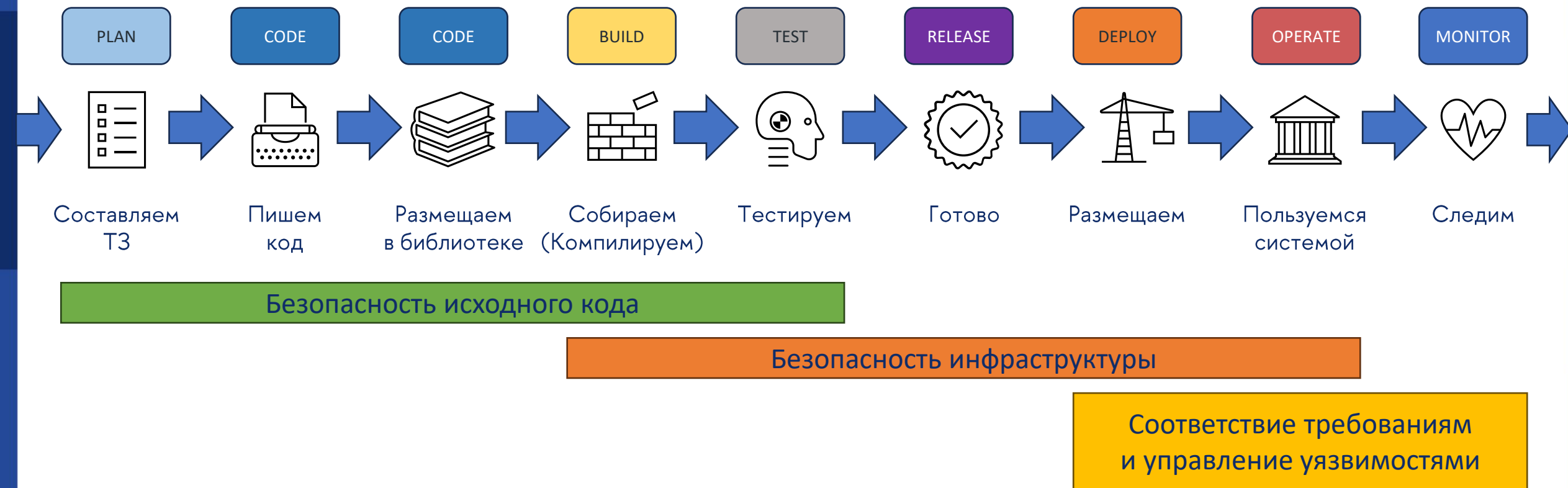
81%

Разработчиков публикуют в репозиториях (то есть считают задачи выполненными) с _известными_ уязвимостями в ПО

96%

Утечек информации происходит по вине самих пользователей (в том числе и разработчиков)

Добавим «безопасность» к процессу разработки ПО



Безопасность исходного кода (PLAN)

PLAN

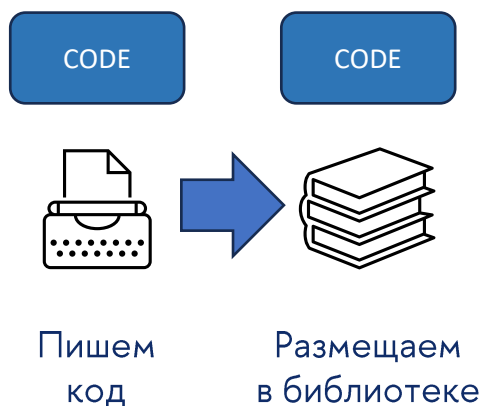


Составляем
ТЗ

Безопасность исходного кода

1. Архитектурные требования
(отказоустойчивость, дублирование, геораспределение)
2. Минимальный базовый уровень требований
3. Обучение группы разработки безопасному процессу
4. Моделирование угроз безопасности

Безопасность исходного кода (CODE)

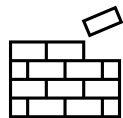


Безопасность исходного кода

1. Разделение кода на ветки и управление версиями
2. Статический анализ кода (линтер)
3. Git-хуки (хотя бы pre-commit для проверки кода на соответствие требованиям/стандартам)
4. Code review – анализ кода другими разработчиками
5. Pull request – запросы на внесение изменений

Безопасность инфраструктуры (BUILD)

BUILD



Собираем
(Компилируем)

Безопасность инфраструктуры

1. Готовые (проверенные) образы контейнеров / виртуальных машин (или операционных систем)
2. Безопасная среда сборки (build) на выделенной инфраструктуре (желательно без доступа в Интернет)

Безопасность исходного кода и инфраструктуры (TEST)

TEST



Тестируем

Безопасность исходного кода

1. Управление учетными записями и паролями
2. SCA – анализ состава ПО (software composition analysis)
3. SAST – статическое тестирование безопасности приложений (на основе исходного кода или бинарного кода, Static Application Security Testing)
4. DAST – динамическое тестирование безопасности приложений (без исходного кода, Dynamic Application Security Testing)
5. Сканирование контейнеров / виртуальных машин

Безопасность инфраструктуры

1. Временные среды для тестирования

Безопасность инфраструктуры (RELEASE)

RELEASE



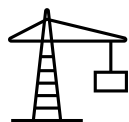
Готово

Безопасность инфраструктуры

1. Управление безопасными конфигурациями (Hardening)
2. Стандартные среды функционирования
3. Репозитории контейнеров / виртуальных машин (единый _проверенный_ источник образов)
4. SBOM – Software Bill of Materials – Подробный список всех используемых библиотек и зависимостей

Безопасность инфраструктуры (DEPLOY)

DEPLOY



Размещаем

Безопасность инфраструктуры

1. Стандартные среды функционирования
2. Операционные системы в режиме безопасных конфигураций (hardened OS)
3. Контроль сетевых подключений
4. Контроль всех элементов инфраструктуры для развертывания
5. Управление секретами (учетными записями от имени которых выполняются сервисы/службы)

Безопасность инфраструктуры и соответствие требованиям (OPERATE)

OPERATE



Пользуемся
системой

Безопасность инфраструктуры

1. Управление доступом пользователей (IAM)
2. Контроль подключаемых клиентов (endpoint)

Соответствие требованиям и управление уязвимостями

1. Управление элементами инфраструктуры (asset)
2. Регулярное сканирование на уязвимости (автоматизированно)
3. Применение (установка) обновлений (патчей)

Соответствие требованиям (MONITOR)

MONITOR

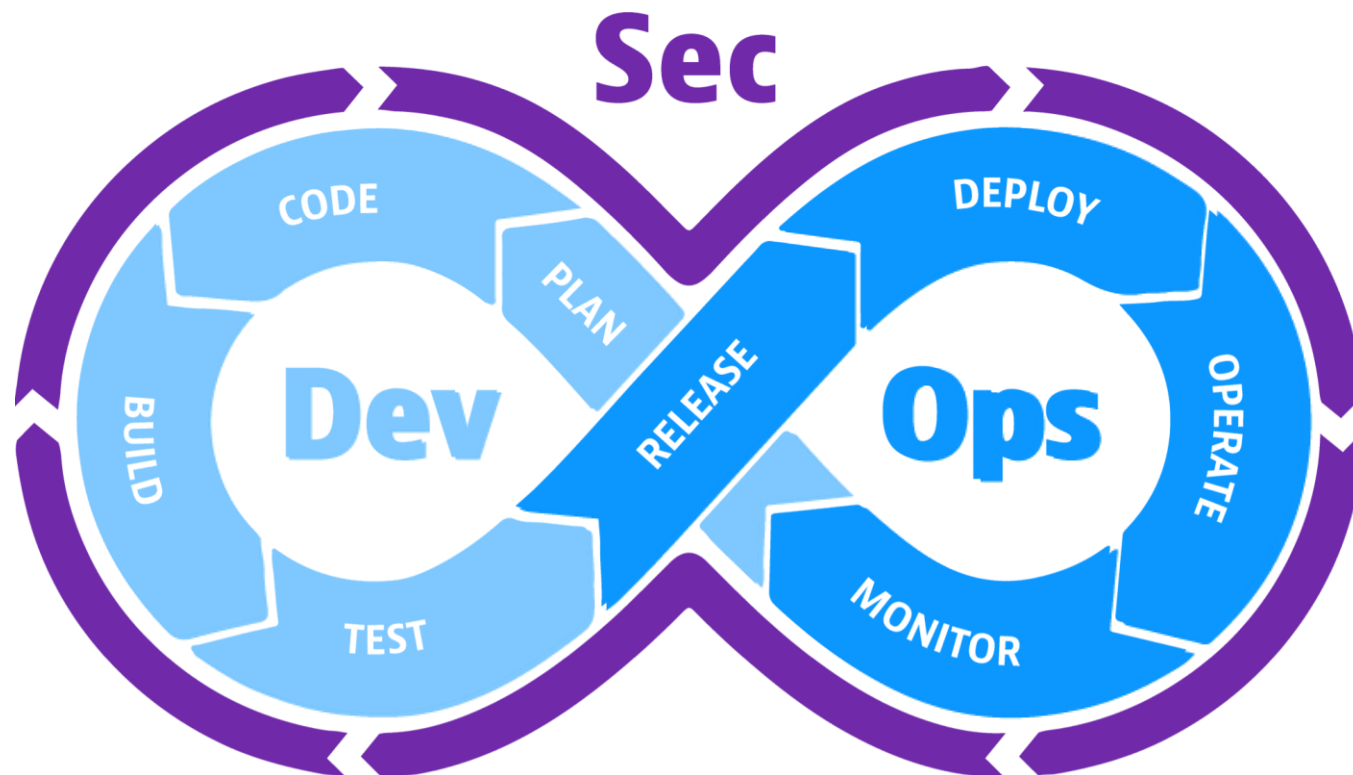


Следим

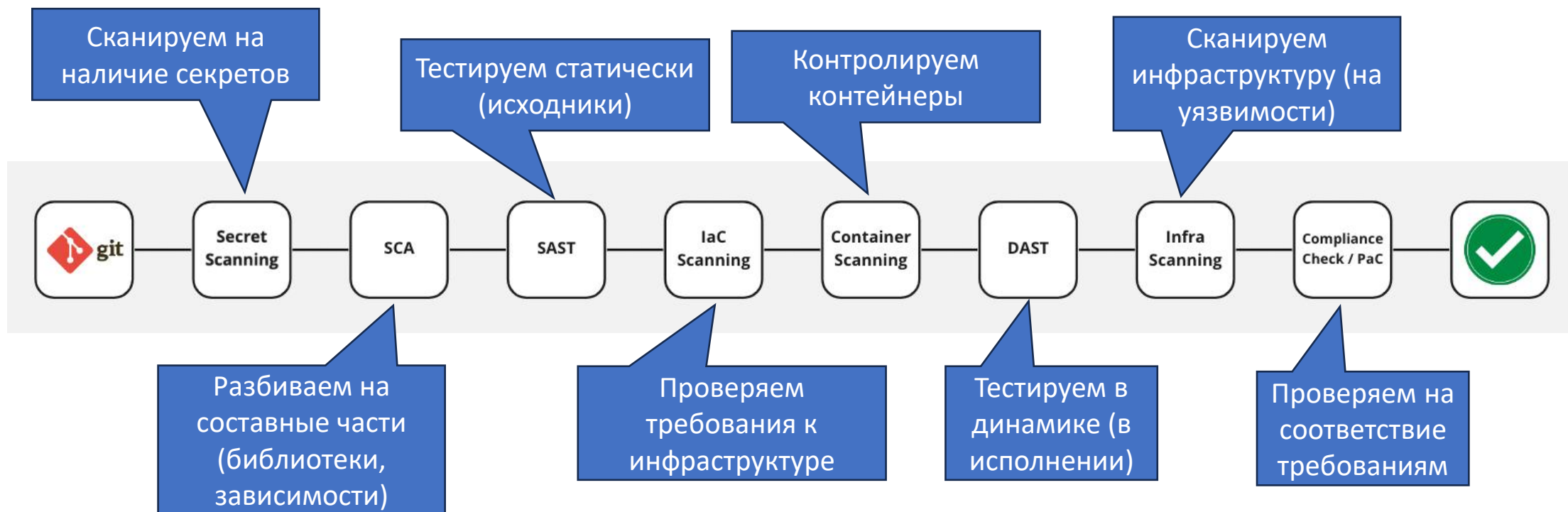
Соответствие требованиям и управление уязвимостями

1. Контроль доступа (журналы, логи приложений) как пользователей, так и потенциальных нарушителей
2. Тестирование на проникновение (pentest)
3. Предотвращение вторжений (intrusion prevention)
4. Аудит (проверка на соответствие требованиям)
5. Интеграция с SIEM (управление инцидентами и событиями)

Development + Security + Operations = DevSecOps



Подход DevSecOps от OWASP



Подход DevSecOps от OWASP

OWASP DevSecOps Guideline

The Open Worldwide Application
Security Project



<https://owasp.org/www-project-devsecops-guideline/>

Инструменты для DevSecOps


JFrog Xray
SCA
sonarqube
SAST
OWASP ZAP
DAST
snyk
SCA **opentext™**
SAST
CheckmarX
SAST / SCA **aqua**
Безопасность
контейнеров **KUBEWARDEN**
Безопасность
контейнеров **OPENVAS**
by Greenbone
DAST /
Сканер уязвимостей

Что будет на семинаре № 11?

- Продолжаем лабораторные работы.
- Поиск уязвимостей. Изучение веб-сервисов.
- Продолжение разбора домашнего задания – все остальные задачи.



